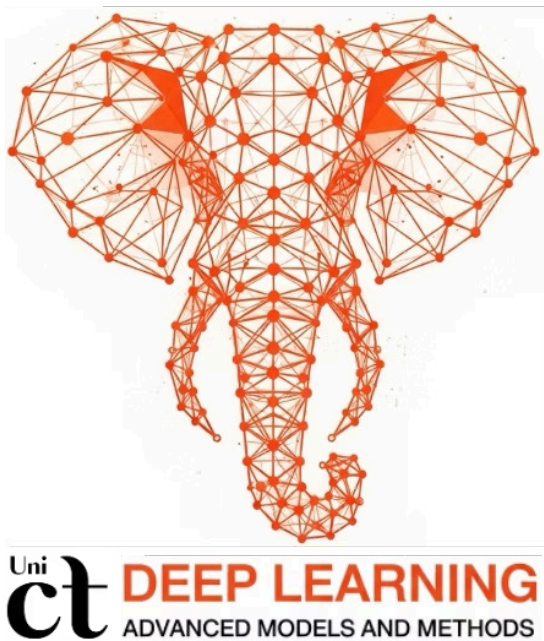




Deep Learning

Advanced Models & Methods

Prof. Antonino Furnari (antonino.furnari@unict.it)

Corso di Laurea Magistrale in Informatica

Dip. di Matematica e Informatica

Università di Catania

Deep Reinforcement Learning

Learning from static data vs learning by doing

Supervised & Self-Supervised Learning

These paradigms thrive on **large, pre-existing, static datasets**. Supervised learning requires explicit labels, while self-supervised methods generate them from raw input. **Data collection is an upfront, often separate process**, leading to immense success.

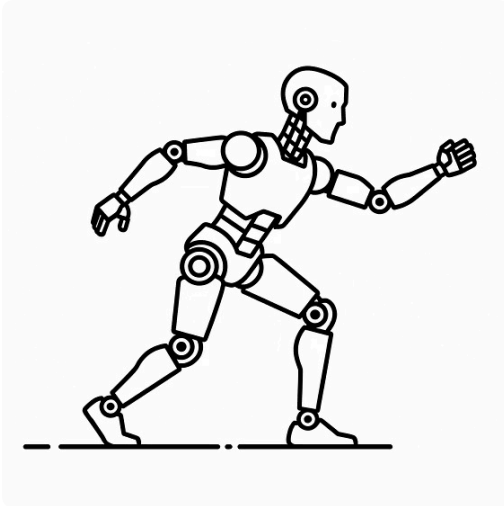
Reinforcement Learning's Approach

RL faces a distinct "data problem" because it **learns by doing**. Data isn't simply collected; it's **generated through continuous interaction**. An agent explores, tries actions, receives immediate feedback from the environment, and adjusts. **Every "mistake" is a valuable lesson.**



Applications: Where RL Shines vs. Supervised Limitations

Reinforcement Learning excels in dynamic environments where direct imitation falls short, actively learning from experience and outcomes.



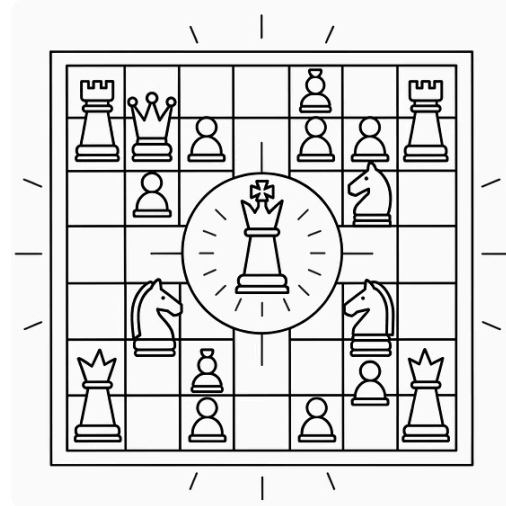
Physical Skills

(e.g., Bike, Robot)

Supervised Approach: Mimic an expert's exact joint angles and telemetry.

Why it Breaks: Covariate Shift: A minor slip or wind gust puts the agent in an unseen state; it panics and crashes.

The RL Solution: Balance
Reward: Agent explores random movements; learns to dynamically recover from failures.



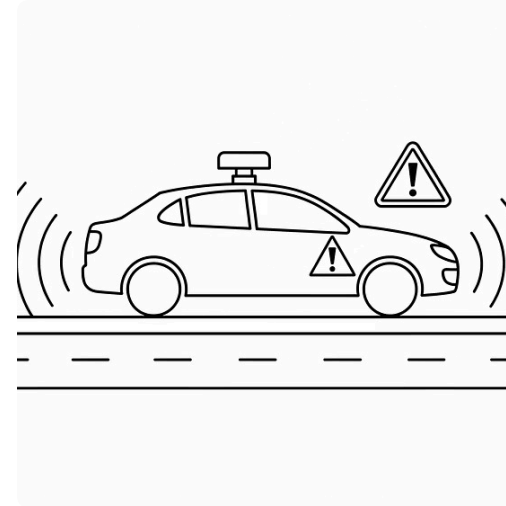
Strategic Games

(e.g., Chess, Go)

Supervised Approach: Clone millions of Grandmaster moves from historical logs.

Why it Breaks: The Novice Trap: Opponent plays a bizarre, sub-optimal move. The state is alien; the model can't look ahead.

The RL Solution: Win
Probability: Plays against itself.
Learns the long-term value of positions, adapting to any strategy.

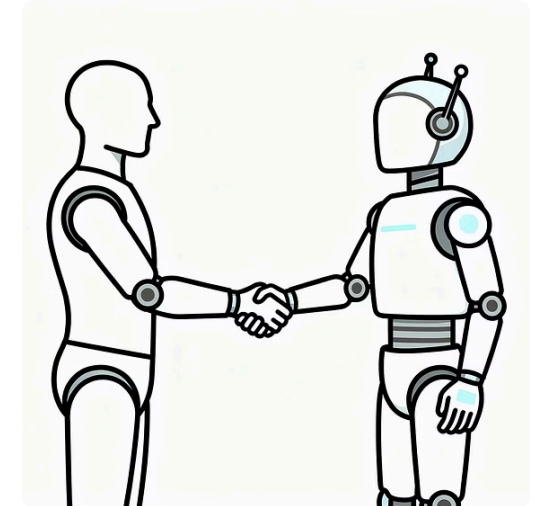


Autonomous Driving

Supervised Approach: Imitate human driver steering wheel angles and braking profiles.

Why it Breaks: Error
Accumulation: A minor 2-inch lane drift puts the car in an unrecorded state; errors compound instantly into a crash.

The RL Solution: Safety & Goal
Reward: Learns from simulated crashes. Evaluates driving quality to actively correct back to safety.



LLM Alignment

(e.g., Safety, Tone)

Supervised Approach: Train on static templates of "polite" human conversations.

Why it Breaks: Sycophancy: Model prioritizes looking agreeable over being accurate; echoes user errors just to match the data pattern.

The RL Solution: Preference
Reward: Evaluates complete candidate responses against a safety/helpfulness scorecard.

Gridworld Example

Let's consider a simple reinforcement example which allows us to introduce some terminology.

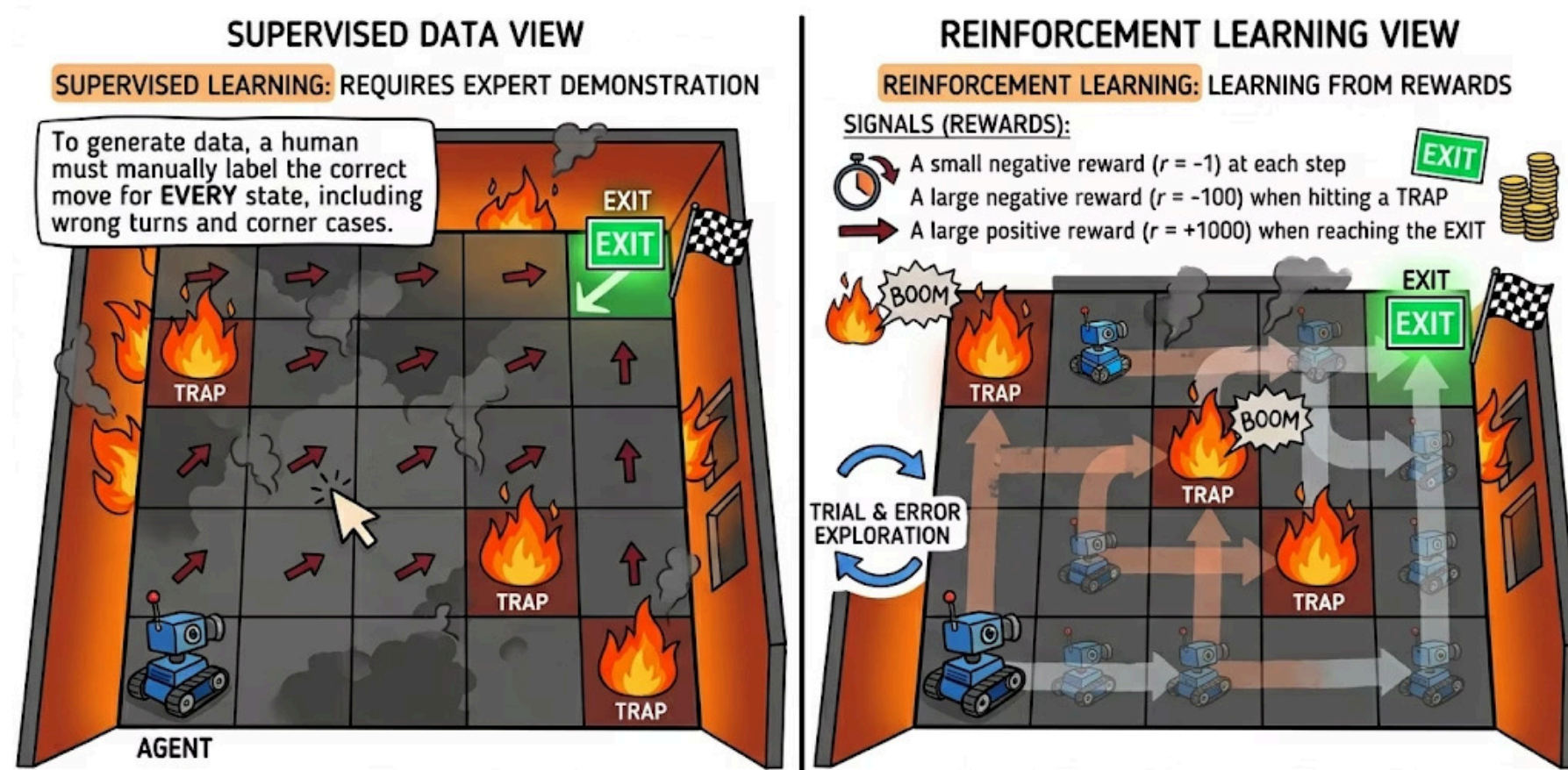
Imagine a robot navigating a grid to find the exit, while avoiding some traps. For example, this could be a building on fire. In each cell, the robot can take discrete actions, such as going up, down, left or right.

We could try to collect supervised data, e.g., by mapping each cell of the grid to its optimal action, but this would require us to work out a solution by hand.

Instead, we let the robot navigate the world and give it a series of signals (rewards):

- A large positive reward when they reach the exit.
- A large negative reward when they reach a trap.
- A small negative reward at each step, to signal that we want them to complete the task quickly.

Reinforcement learning allows to learn how to move in this gridworld by leveraging these signal, without explicit supervision.

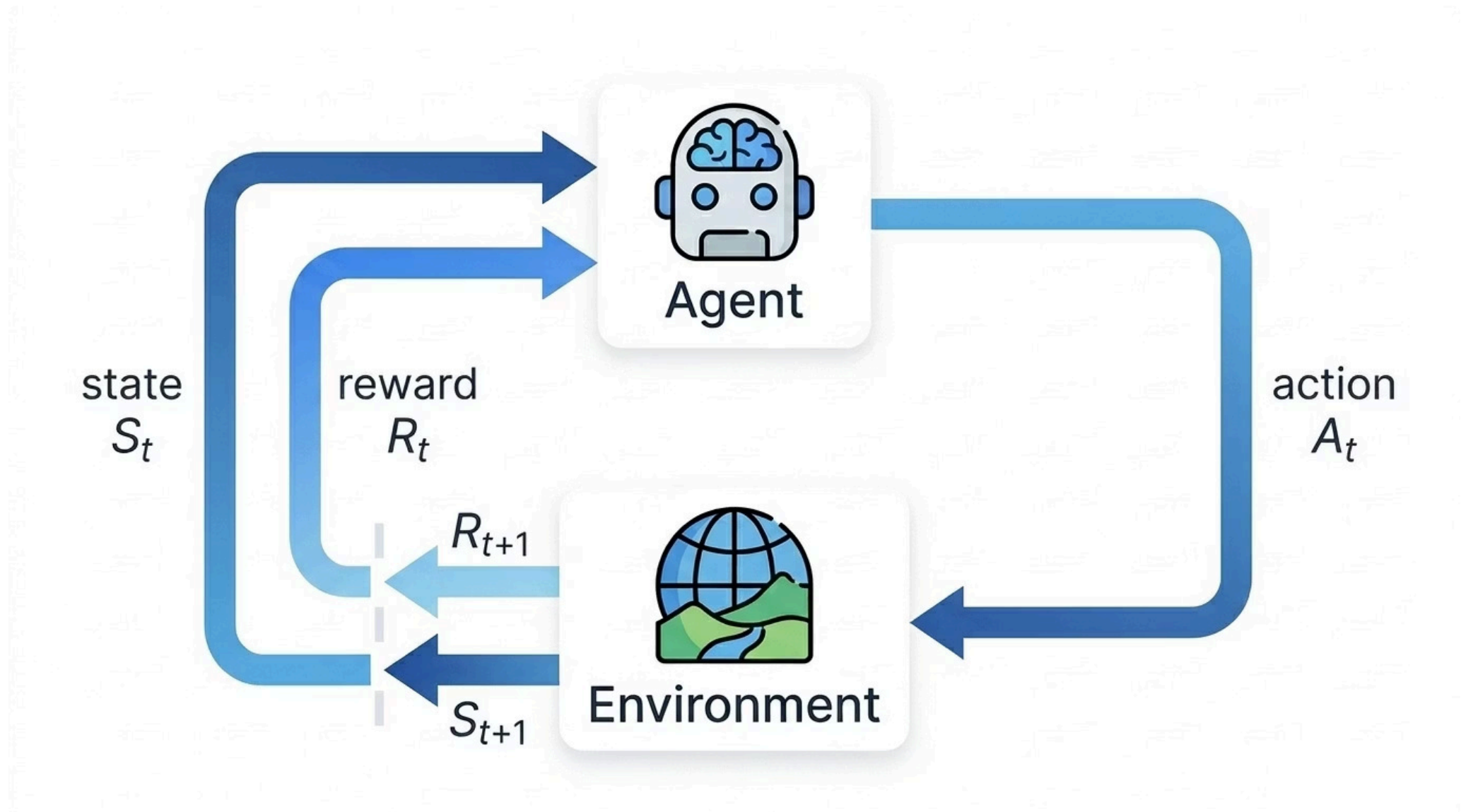


Gridworld Terminology

- **Agent** — the robot navigating the grid — the learner that takes actions and receives feedback
- **Environment** — the grid itself, including walls, obstacles, and the goal — everything external to the agent
- **State** ($s_t \in \mathcal{S}$) — the agent's current cell (row, column) — each cell is a distinct, discrete state
- **Observation** ($o_t \in \mathcal{O}$) — what the agent perceives from its current cell — may be the full grid or just nearby cells depending on the setting
- **Action** ($a_t \in \mathcal{A}$) — move up, down, left, or right — a discrete set of four possible moves at each timestep
- **Reward** (r_t) — +1 for reaching the goal; -1 for hitting an obstacle; -0.01 per normal step — small negative step costs discourage unnecessarily long paths
- **Policy** ($\pi : \mathcal{O} \rightarrow \mathcal{A}$) — the rule that tells the agent which direction to move from each cell — the agent's learned navigation strategy
- **Return** (G_t) — the total discounted sum of rewards from the current cell to episode end — captures whether the agent found a short, safe path to the goal
- **Credit assignment problem** — if the agent reaches the goal after 20 steps, which moves actually mattered? — rewards are only given at the end, making it hard to identify which early moves were good or bad

The Reinforcement Learning Cycle

Reinforcement learning is usually depicted as a cycle:



Here, the agent learns to take actions A_t in an environment, transitioning from state S_t to state S_{t+1} and generating a reward R_{t+1} .

Episodic vs. Continuous Reinforcement Learning

Episodic RL: Defined Tasks

- Interaction unfolds in **distinct episodes** with clear start and end points.
- Agent's goal is to maximize total reward within **each episode**.
- Environment often resets after each episode (e.g., game ends, maze solved).

Examples: Playing a chess game, completing a level in a video game, robot navigating a specific route.

Continuous RL: Ongoing Interaction

- Interaction is **ongoing and indefinite**, without a clear termination state.
- Agent's goal is to maximize reward per unit of time or cumulative reward over an **infinite horizon**.
- Environment typically does not reset unless manually intervened.

Examples: Self-driving car in traffic, robotic cleaning, stock market trading agent.

Goals and Returns

The agent's objective is to maximize the **expected cumulative return** over time. The **discounted return** at timestep t is defined as:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

The discount factor $\gamma \in [0, 1]$ controls the agent's temporal horizon and serves two critical purposes:

$\gamma \rightarrow 0$: Myopic Agent

Only the immediate next reward matters. The agent is purely greedy and ignores long-term consequences — dangerous in environments requiring planning.

$\gamma \rightarrow 1$: Far-Sighted Agent

All future rewards are weighted nearly equally. Enables long-horizon planning, but requires the sum to converge — guaranteed only for episodic tasks or $\gamma < 1$.

Mathematical Necessity

Discounting ensures the infinite geometric series converges (especially in continuous RL). Without it, the return is unbounded for continuing tasks, making optimization undefined.

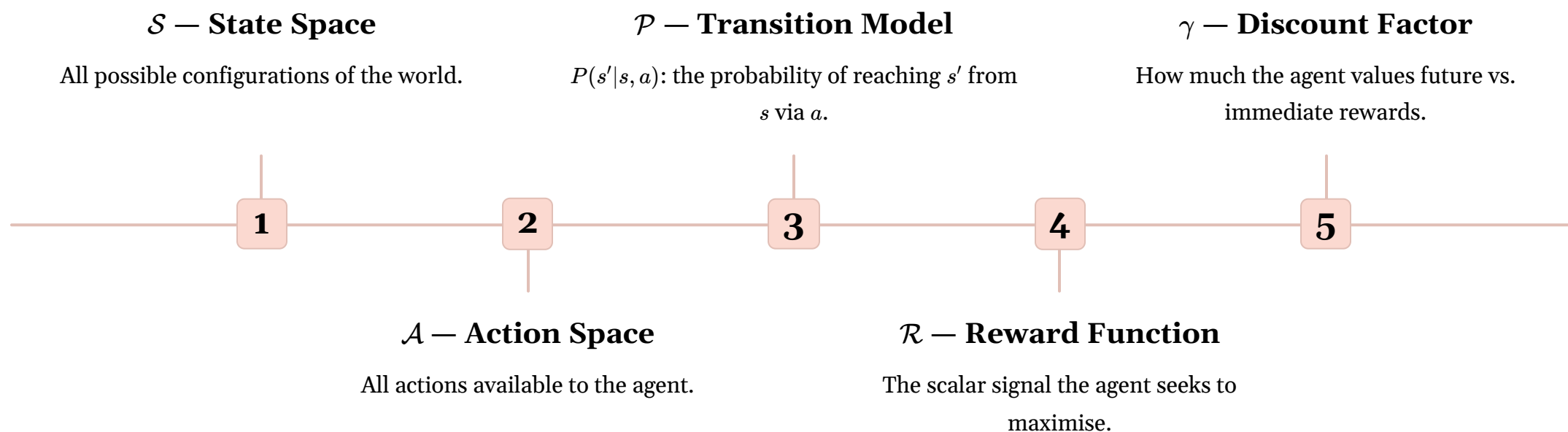
Epistemic Intuition

Uncertainty about the future grows with time. A reward 100 steps away is inherently less certain than a reward next step — discounting reflects this rational hedging.

The Markov Decision Process (MDP)

The MDP is the formal mathematical framework underlying all RL. It provides a precise language for the agent–environment interaction and enables principled algorithm design. We see a Markov Decision Process as a tuple:

$$\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$$



The Markov Property

Markov Decision Processes assume the **Markovian property** is valid, i.e.,

$P(s'|s, a)$ depends *only* on the current state — not the history, i.e.,

$$P(s_t|s_{t-1}, a) = P(s_t|s_{t-1}, s_{t-1}, \dots, s_0, a)$$

This is **true in many cases** (e.g., think of chess, where the current position of the board is what matters) and we can usually define a state space that effectively embeds the trajectory of past states so that the property is valid.

For instance, **in a driving simulator** the raw observation might only include the current camera frame, which does not reveal the car's velocity. The process is therefore not Markovian. But if **we define the state as the last few frames** (e.g., the last 4 images), then this augmented state contains enough information to infer motion, making the transition effectively Markovian.

Partially Observable Markov Decision Processes (POMDPs)

In practice, deep RL agents often face **POMDPs** (Partially Observable MDPs), where the state is not fully observed. For instance, rather than knowing everything about a car's position, velocity, mass, etc., we observe pixels (e.g., video frames). While we are not observing the state s_t directly, we can still obtain observations o_t and infer belief states from observations which are generated according to a model $P(o_t|s_t)$.

Policies

Under this probabilistic framework, we will see a policy as a function telling us what is the probability of taking an action a in a state s :

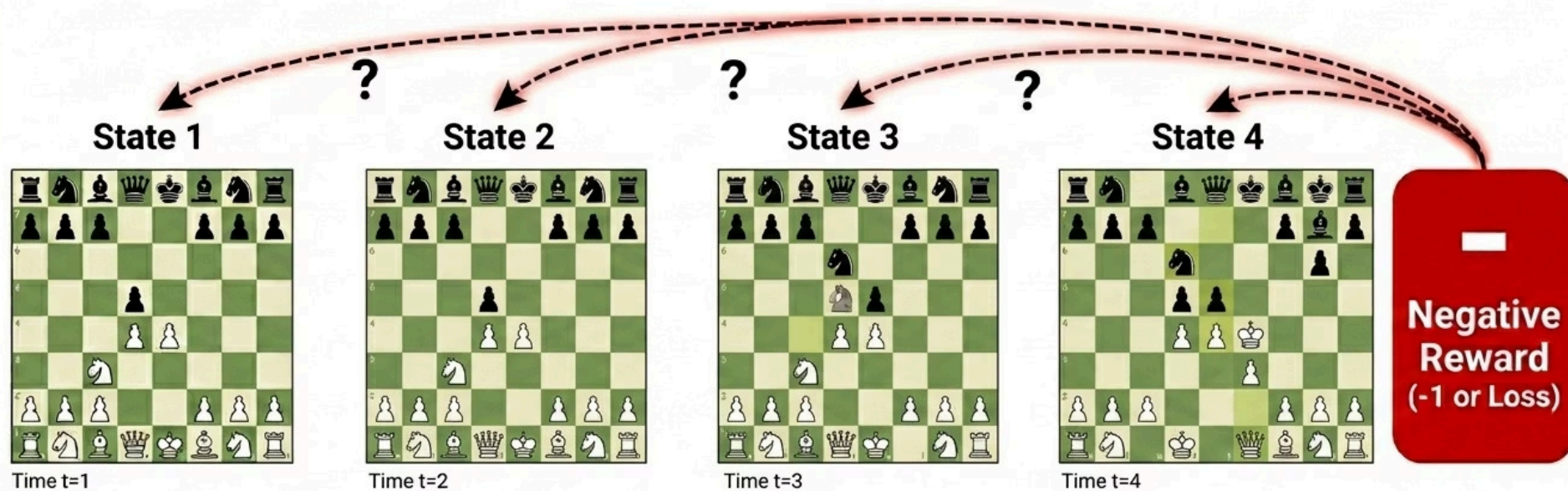
$$\pi(a|s) = P(a|s)$$

Once we have such a model, we can effectively take action a^* in state s as follows:

$$a^* = \arg_a \max \pi(a|s)$$

The Credit Assignment Problem

One of the central challenges in Reinforcement Learning is the **Credit Assignment Problem**. It refers to the difficulty of determining which specific actions, among a long sequence of actions, are responsible for a given reward or penalty, especially when rewards are sparse and delayed.



1

Delayed Rewards

The agent often receives feedback (reward) much later than the actions that caused it. For example, a chess player only gets a reward at the end of the game, not for every move.

2

Sparse Rewards

Rewards are not given after every action, or even frequently. They might only appear for critical achievements or at the end of an episode, making it hard to connect them to preceding actions.

3

Long Action Sequences

A single outcome can be the result of a complex series of choices and actions. Identifying the causal links between individual actions and the final outcome becomes a non-trivial task.

This problem makes learning difficult because the agent needs to understand which actions **truly contribute** to maximizing long-term return, not just immediate gratification. It requires sophisticated methods to propagate credit (or blame) back through time to the relevant actions.

Value Functions and Q-Functions

Value functions compress the agent's knowledge of long-term returns into **a single scalar per state** (or state-action pair). They are the core abstraction that enables efficient credit assignment.

State Value Function $V_\pi(s)$

"How good is it to be in state s , following policy π ?"

Formally: the **expected return starting from state s and thereafter following π** :

$$V_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

Action-Value Function $Q_\pi(s, a)$

"How good is it to take action a in state s , then follow policy π ?"

Formally: the expected return starting from state s , taking action a and then following policy π :

$$Q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

These two quantities are related:

$$V_\pi(s) = \sum_a \pi(a|s) Q_\pi(s, a)$$

❏ **Insight:** If we know the optimal $Q^*(s, a)$, the optimal policy is trivial — always select $\arg \max_a Q^*(s, a)$. No explicit policy representation needed.

Computing V and Q functions - Monte Carlo Baseline

The definitions we gave above for state value and action value (or Q-) functions involve the computation of expectations. But how do we compute them?

Monte Carlo Baseline: run multiple episodes following a policy π , we collect rewards and store them either indexed by states or by state-action pairs. Then we compute expected rewards in a given state or state-action pairs by averaging all returns observed from that state (or state-action pair).

Monte Carlo estimates are unbiased: with enough episodes, they converge to the true value functions. However, they are also impractical for policy learning. Each time we change the policy, the value estimates become invalid, and we must run many new episodes to recompute them from scratch.

Optimal Value Functions

Solving a reinforcement learning problem means finding a policy π that will receive a lot of reward over time. This means visiting states with high values or taking actions maximizing the Q-Function. To reason about that, we can define optimal Value and Q-Functions:

$$V_*(s) = \max_{\pi} V_\pi(s) \quad Q_*(s, a) = \max_{\pi} Q_\pi(s, a)$$

These are in practice the values we would get if we had the "oracle" best policy ever.

The Bellman Equation — The Engine of RL

The Bellman Equation provides a recursive framework to update value and Q-functions based on the immediate rewards we obtain when transitioning from a state to another state, without having to wait for the end of an episode to perform Monte Carlo estimates.

Bellman Expectation Equation for Values

Assume we are following a policy π which let us transition from state s at time t to state s' at time $t + 1$ receiving a reward R_{t+1} . The Bellman Expectation Equation is defined as follows:

$$V_{\pi}(s) = \mathbb{E} [R_{t+1} + \gamma V_{\pi}(s') | S_t = s]$$

In practice, the Bellman Equation is telling us that

— The value of a state is the expected immediate reward (R_{t+1}) plus the discounted value of the next state.

To see why this makes sense, let's try to unroll the recurrence:

$$\begin{aligned} V_{\pi}(s_t) &= \mathbb{E}[R_{t+1} + \gamma V_{\pi}(S_{t+1}) | S_t = s_t] = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 V_{\pi}(S_{t+2}) | S_t = s_t] = \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s_t\right] = \\ &\quad \mathbb{E}[G_t | S_t = s_t]. \end{aligned}$$

This shows that the Bellman equation is a recursive way of expressing the expected return. As we will see, this gives us the theoretical framework to build iterative methods that, rather than waiting for the end of episodes to compute value functions, can update their values using immediate rewards.

Bellman Expectation Equation for Q-functions

The same principle applies to Q-functions. In this case, the corresponding Bellman Equation is:

$$Q(s, a) = \mathbb{E}[R_{t+1} + \gamma V_{\pi}(S_{t+1}) | S_t = s, A_t = a]$$

This works exactly as the Bellman expectation equation for values, but it is conditioned on the action taken at time t .

From Monte Carlo to Temporal-Difference Learning

Monte Carlo methods estimate value functions using full returns, but this is often slow and impractical for online learning. The Bellman expectation equation suggests a more efficient approach: **bootstrapping** from the value of the next state.

Temporal-Difference (TD) learning leverages this by updating value estimates using the immediate reward and the current estimate of the next state's value, allowing for continuous, incremental adjustments without waiting for an episode to end.

Temporal-Difference Update Rule

The temporal-difference update rule is defined as follows:

$$V(s_t) \leftarrow V(s_t) + \alpha [R_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

$R_{t+1} + \gamma V(s_{t+1})$ is the TD target.

This is the estimated total return from the current state, combining the immediate reward with the discounted, estimated value of the next state. The Bellman equation tells us that this is "a good target" as it converges to the return.

$R_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ is the TD error.

This difference quantifies how "surprised" the agent is by the actual immediate reward plus the estimated value of the next state, compared to its current estimate of the current state's value. It answers the question "how better is my return gonna be if I go from state s_t to state s_{t+1} ?"

We move $V(s_t)$ a little toward the target.

The learning rate α determines the step size of this adjustment, ensuring a gradual refinement of the value function based on new experiences. In practice, we update the old value $V(s_t)$ with the difference between the target and old value, mediated by the learning rate.

This is the first time we use the Bellman equation not just as a definition, but as a practical **update rule** that forms the foundation for many powerful RL algorithms.

From Evaluation to Control: Finding the Best Policy

So far, we've focused on **policy evaluation**—understanding how good a given policy is. However, our ultimate goal in Reinforcement Learning is not just to evaluate, but to **control** the agent's behavior by finding the optimal policy.

This means we want to discover the policy that maximizes the total expected reward. This optimal behavior is characterized by the **Bellman Optimality Equations**:

$$V_*(s) = \max_a Q_*(s, a) \quad Q_*(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} Q_*(S_{t+1}, a')]$$

These equations describe the values under an optimal policy. Critically, **if we can accurately estimate Q_* , the optimal policy is trivial**: simply choose the action that yields the highest Q-value in any given state.

Tabular Q-Learning — A Concrete Foundation

Building upon Temporal-Difference learning, Q-Learning is a pivotal algorithm for finding optimal policies in Reinforcement Learning. It applies TD principles directly to the Bellman Optimality Equation, allowing an agent to learn the best actions in any given state without requiring a model of the environment.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

Similarly to the case of values:

$r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a')$ is the target

This is the immediate reward + the discounted optimal q-value (if we were to take the best action). The Bellman optimality equation tells us this is a good target.

$r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)$ is the error

Quantifies how surprised we would be if we were to take the best action and transition to another state.

As in TD, we are moving a little towards the target

We move towards the target using the learning rate α as in TD learning.

1 Bootstrapping

Similar to TD learning, Q-Learning leverages bootstrapping. It updates the value of a state-action pair using its own estimate of the maximum future Q-value from the next state, rather than waiting for an entire episode to conclude.

3 Model-Free Learning

Q-Learning is a model-free algorithm. This means it learns the optimal Q-function directly from interactions with the environment, without needing to know or estimate the environment's transition probabilities or reward function.

2 Optimality through Maximization

The $\max_{a'}$ operator is what drives Q-Learning towards an optimal policy. It considers the best possible Q-value for the next state, effectively learning the value of the optimal action, regardless of what action was actually taken.

4 Off-Policy Control

Q-Learning is an off-policy algorithm. It learns the optimal Q-function while potentially following an exploratory policy that visits many state-action pairs. It can discover the optimal policy even if it's not directly executing it.

The Exploration-Exploitation Dilemma

As agents learn through interaction, they face a fundamental trade-off: **should they exploit their current knowledge to maximize immediate rewards, or explore the environment to discover potentially higher future rewards?**

The Need for Experience

Q-Learning, like other reinforcement learning algorithms, relies on collecting experiences from the environment. The agent needs to try actions and observe rewards and next states to update its Q-value estimates.

The Greedy Pitfall

If the agent always chooses the action with the highest current Q-value (a purely greedy policy), it risks getting stuck in suboptimal strategies. Early in training, Q-values are often inaccurate, and acting greedily can prevent the agent from discovering better paths.

Introducing ϵ -Greedy

To balance exploration and exploitation, the ϵ -greedy policy is commonly used. With probability $1 - \epsilon$, the agent chooses the action with the highest Q-value (exploits). With probability ϵ , it chooses a random action (explores).

This strategy ensures that the agent occasionally tries new actions, even if they don't seem optimal at first, allowing it to improve its understanding of the environment and potentially find better solutions over time. Typically, ϵ starts high and decays over training.

Note. Q-Learning is off-policy: this means that while we can follow a policy to collect evidence, the update formula always uses a max operation, effectively leading to greedy approach to update q values.

Q-Learning in a Gridworld: A Concrete Example



Grid World Q-Learning Demo

From Tables to Networks: Deep Q-Learning (DQN)

The conceptual leap from tabular RL to deep RL is both elegant and consequential. We simply parameterize the Q-function with a neural network:

$$Q(s, a) \approx Q_{\theta}(s, a)$$

This also naturally allows another leap: rather than taking as input "states", we can consider "observations" and learn instead $Q_{\theta}(o, a)$.

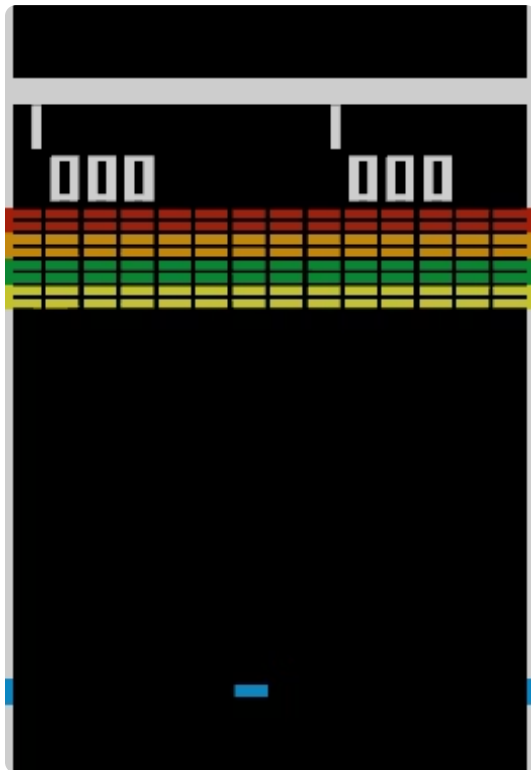


Image source: [wikipedia](#)

Atari Game example

Imagine an agent learning to play the Atari "breakout" game.

Actions: the agent can perform the following actions:

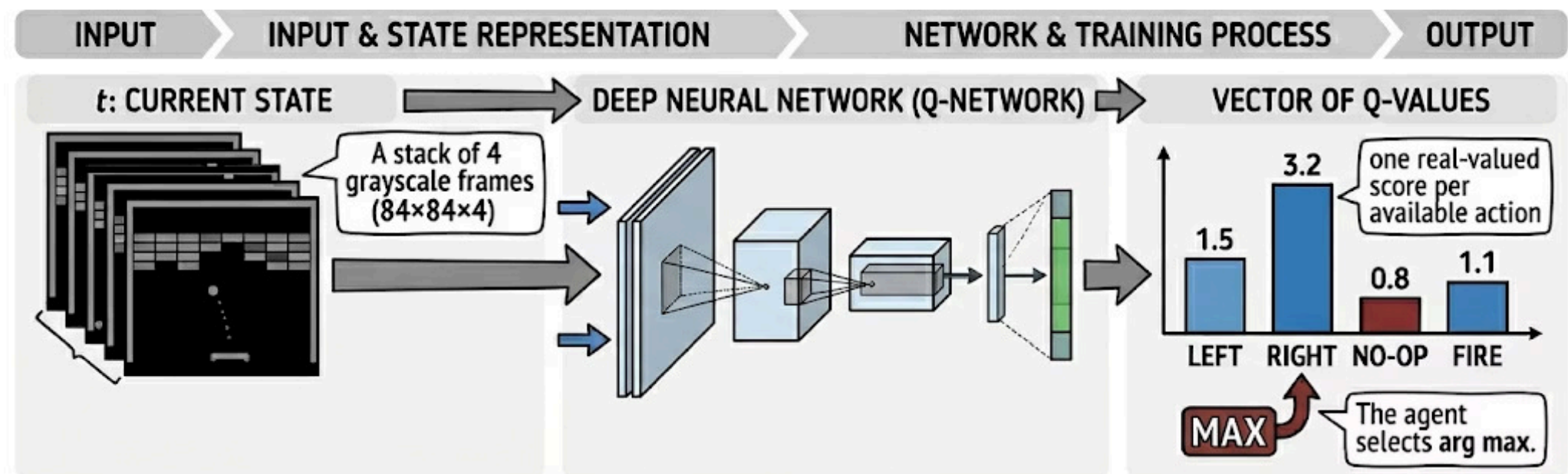
- NOOP (No Operation) – The paddle stays completely still.
- FIRE – Launches the ball to start the game or resume after losing a life.
- RIGHT – Moves the paddle to the right.
- LEFT – Moves the paddle to the left.

Environment: a simulator receiving actions from the agent and returning images of the game with rewards

Observations: while we cannot model the internal state directly (e.g., cart position, ball angle, etc.), we can work directly with image observations. For instance, we can take as input a $84 \times 84 \times 4$ stack of 4 grayscale frames.

Rewards: the simulator assigns a reward. For instance:

- +1 to +7 points each time the ball destroys a brick (higher tiers score more)
- 0 reward for keeping the ball in play without hitting a brick
- -1 (or episode termination) when the agent drops the ball and loses a life



The Architecture Mapping

Input: A stack of 4 grayscale frames ($84 \times 84 \times 4$). For robotics, a vector of joint angles and velocities.

Output: A vector of Q-values — one real-valued score per available action.

Playing: We sample the most recent four frames, we feed them to the network and we choose the action giving the largest q-value (a greedy policy).

The Critical Advantage: Generalization

This is the transformative property that tables cannot offer. When the network updates its weights after visiting state s , those weights also shift the predicted Q-values for every state s' that activates similar features.

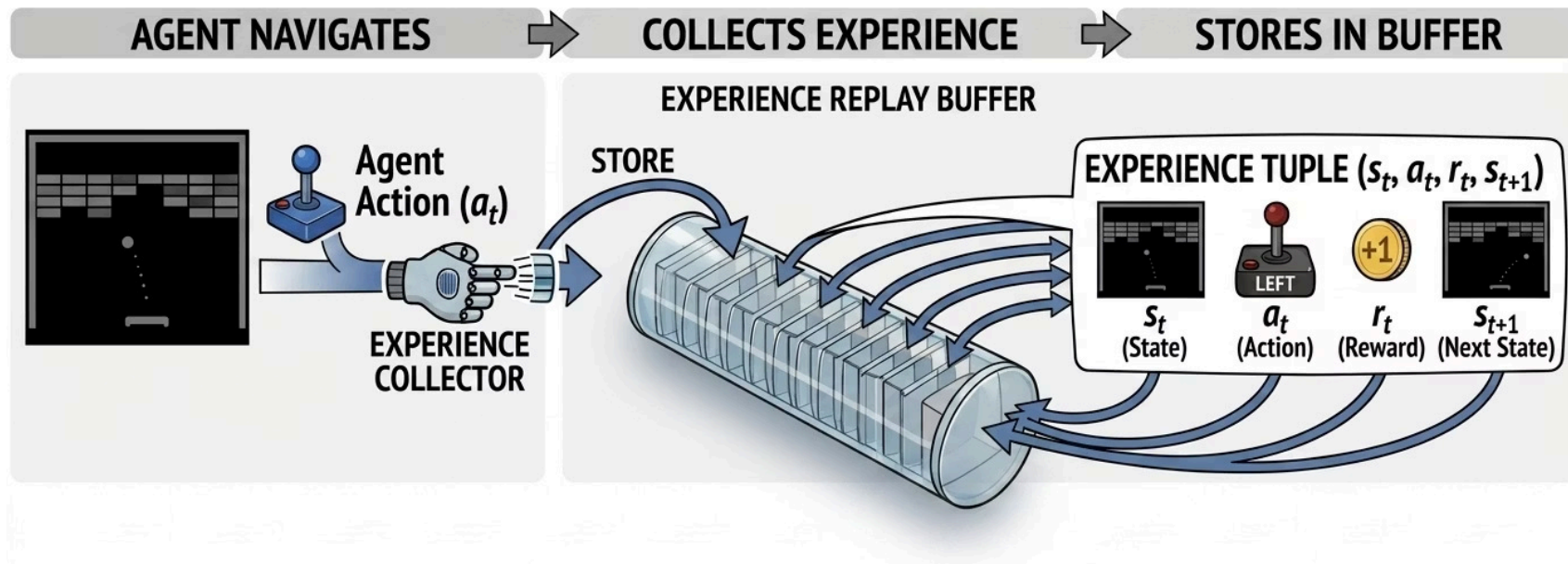
The network learns **reusable representations** — edges, objects, spatial layouts — that transfer across novel states never seen during training.

How Does a DQN Learn?

The Deep Q-Network (DQN) combines reinforcement learning with deep neural networks to approximate the optimal Q-function. Its learning process involves two key aspects:

1. Sampling Observations

The agent navigates the environment by taking actions and collecting experience tuples of the form (s, a, r, s') — state, action, reward, next state. These tuples are stored in a buffer and are our training data (often called "experience"): they record what the agent observed, what it did, what reward it received, and where it ended up.



2. Training Criterion

The DQN is trained to minimize the difference between its predicted Q-values and a "target" Q-value. This target is derived from the Bellman equation, representing the best possible cumulative future reward for a given state-action pair.

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim U(D)} \left[\left(r + \gamma \max_{a'} Q(s', a'; \bar{\theta}) - Q(s, a; \theta) \right)^2 \right]$$

Here:

- $U(D)$ denotes a uniform random sample from the buffer of data D (more of this in a moment):
- The loss function is the Mean Squared Error (MSE) between
 - the current Q-value prediction, $Q(s, a; \theta)$, and
 - the "target" value, $r + \gamma \max_{a'} Q(s', a'; \bar{\theta})$.

Note: the prediction and target networks have different parameters. We will see why and how this works in the next slides.

The network's weights (θ) are updated via gradient descent to reduce this error.

Where does the loss function come from?

Remember the Q-learning rule:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

If you think about it, the learning criterion is exactly the same, but we are doing it with Gradient Descent, rather than with temporal differencing.

The Deadly Triad

Why Deep RL is Unstable

Sutton & Barto identify a fundamental source of instability unique to deep RL. Algorithms tend to **diverge** — Q-values explode to infinity — when all three of the following are present simultaneously:

① Function Approximation

Neural networks generalize across states — a weight update for one state perturbs predictions for all related states. This introduces systematic, uncontrolled errors.

② Bootstrapping

Bellman updates use the network's own predictions as targets — a circular dependency. Errors in the target compound with each update step rather than washing out.

③ Off-Policy Learning

Training on stored transitions (a replay buffer) that were collected under a different, older policy. The data distribution diverges from the current policy, amplifying instability.

DQN's two key innovations — Experience Replay and Target Networks — are specifically designed to break two of these three conditions.

Solution 1: Experience Replay

If we were to train the algorithm sequentially in a naive way, without sampling experiences from a buffer, nothing would work. Indeed, the most basic assumption of stochastic gradient descent — **that training samples are independently and identically distributed** — is catastrophically violated by RL's sequential data stream. State s_t and state s_{t+1} are nearly identical; training on them consecutively amounts to running gradient descent on the same example repeatedly.

The Mechanism

Every transition (s, a, r, s') is stored in a large **ring buffer** (replay memory) of fixed capacity — typically 1 million transitions for Atari.

At each training step, a **random mini-batch** of transitions is sampled uniformly. The sequential correlation is broken: the batch may contain transitions from 10,000 timesteps ago alongside recent ones.

Two Key Benefits

- **Decorrelation:** Randomly sampled batches approximate i.i.d. samples — SGD's assumptions are (approximately) restored
- **Sample Efficiency:** Each transition can be replayed and learned from many times before being evicted — critical when environment interaction is expensive

❏ **Trade-off:** Replay buffers enable off-policy learning, which is the third component of the Deadly Triad. They solve correlation but retain the off-policy instability.

Solution 2: Target Networks

In a naive implementation, we could use the same weights θ for both the prediction and target:

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim U(D)} \left[\left(r + \gamma \max_{a'} Q(s', a'; \bar{\theta}) - Q(s, a; \theta) \right)^2 \right]$$

with $\bar{\theta} = \theta$ and both updated with gradient descent.

However, that often leads to catastrophic divergence, as every gradient step shifts the target, creating a feedback loop where the network perpetually chases a moving goal.

The Fix: Freeze the Target

Maintain a separate **target network** with parameters $\bar{\theta}$, a **periodic copy** of the main network. Targets are computed using $\bar{\theta}$, which is **frozen** for N steps (e.g., every 10,000 steps).

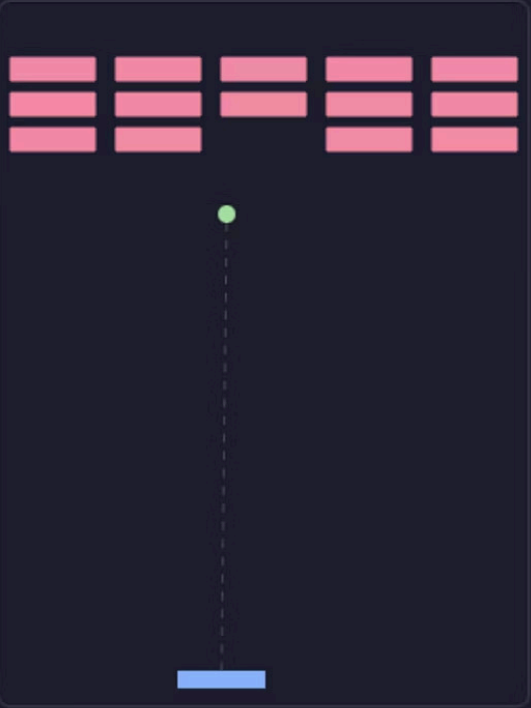
$$Y = r + \gamma \max_{a'} Q(s', a'; \bar{\theta})$$

The target is now stationary within each update window — gradient descent has a fixed objective to minimize.

Deep Q-Learning Demo

Deep Q-Learning Demo

Environment



Neural Network



Nodes represent activations (ReLU).
Green connections = Positive Weights. Red = Negative.

Training Controls

Pause

Simulation Speed 1x

Higher speed executes multiple logic steps per rendering frame, allowing the agent to learn much faster.

Hyperparameters

Replay Buffer	✓
Target Network	✓
Target Goal (Avg Reward)	15
Target Update Freq	200 steps
Batch Size	32
Learning Rate	0.01
Gamma (Discount)	0.95

Beyond Deep Q-Learning

While DQN revolutionized value-based methods, the field of Deep Reinforcement Learning encompasses a broader spectrum of powerful approaches, particularly for complex action spaces and continuous control.

Policy-Based Deep RL

Directly optimizes the agent's decision-making policy, which is a mapping from states to actions, often outperforming value-based methods in continuous action spaces.

Actor-Critic Architectures

Combines policy-based (Actor) and value-based (Critic) methods. The Actor suggests actions, and the Critic evaluates them, providing more stable and efficient learning.

Proximal Policy Optimization (PPO)

A popular, robust, and relatively simple algorithm that balances stability with sample efficiency, widely used in robotics, gaming, and other complex control tasks.

GRPO (Group Relative Policy Optimization)

An advanced algorithm designed for multi-agent reinforcement learning, focusing on optimizing policies based on the relative performance within a group of agents.